

Product Requirements Document [PRD] - Netflix Clone Project

Title/Initiative	Netflix Clone Project
Date & Version	9/15/2026 Ver. 1
Product - Point of Contact (POC)	Manuel Rosales
Design POC	Manuel Rosales
Tech POC	Manuel Rosales
Marketing POC	N/A

Why? (Objective)

This document describes a portfolio artifact, not a commercial product. The goal is to prove hands-on systems engineering ability by rebuilding a recognizable product's architecture under a hard cost constraint. Scope, metrics and trade-offs throughout are written against that goal — not against a hypothetical global user base.

1. **Outcome for the author (the "business"):** Produce verifiable evidence of distributed-systems and cloud skill that certifications and coursework cannot demonstrate — a system designed, deployed and operated end to end. A secondary outcome: the document itself evidences product-management thinking, since the cost constraint forced real trade-offs rather than a feature wishlist.
2. **Outcome for viewers (~10 invited testers):** Browse a catalog of movies and TV shows drawn from multiple regions, sign in, maintain per-profile favorites, and play trailers — a working product, not a mock-up.
3. **The constraint that shapes everything below:** A fully AWS-hosted version was priced and rejected. ECS Fargate alone reached roughly \$12 per three weeks for a single small task, before a load balancer (~\$11 per three weeks) or NAT gateway (~\$23 per three weeks) — and an EKS control plane bills hourly regardless of traffic. For a system serving ten testers, that spend buys nothing. The architecture is therefore **hybrid cloud**: the application runs on a self-hosted cluster, and only services already provisioned in AWS remain there.

How do we measure success?

1. **Objective: demonstrate production-grade architecture at near-zero cloud cost.**
 1. **KR1 (primary — cost):** Total monthly AWS spend stays under **\$30**, with **RDS and the bastion EC2 as the only billed services**. No new billed AWS service is introduced without displacing an existing one. This is the headline result, not a footnote: the project's central claim is that the architecture was achieved without renting it.
 2. **KR2 (resilience):** With the cluster live, a worker node can be taken offline and the application keeps serving — Kubernetes reschedules pods and the load balancer stops routing to the dead node, with no visitor-facing error. *Note: this proves node-level redundancy only. There is no multi-AZ or multi-region redundancy in this design, by choice.*

3. **KR3 (reproducibility):** The application deploys to the cluster from manifests, and a bad release rolls back to the previous revision without manual repair.
2. **Guardrails** — metrics that must not degrade while pursuing the above:
 1. Monthly cloud spend under **\$30**. Costs that bill *hourly regardless of traffic* are the specific thing being avoided; this is why the compute layer moved off AWS.
 2. TMDb terms of use are respected: no profit from the deployment, required attribution present, and their content is not used to train or validate ML models.
 3. User data handling follows applicable data-governance requirements, and a privacy policy page is published.
3. **Deliberately not measured:** throughput, concurrent-user scale, and per-region latency. With ~10 invited testers and no public launch, such figures would be manufactured rather than observed. Claiming them would weaken the document, not strengthen it.

Who are the users:

1. **Persona 1 — The viewer (invited tester).** Someone who wants a curated collection of international film and television for cultural immersion, rather than an algorithmically narrowed feed of domestic content.
 - *Job to be done:* "When I want to explore films from outside my own country, I want to browse a catalog organized by region, so I can find something I would never have been shown otherwise."
2. **Persona 2 — The evaluator (hiring manager, interviewer, or QA-minded reviewer).** The persona this project most directly serves. They are assessing whether the author can design, deploy and operate a real system, and will probe the reasoning behind each choice.
 - *Job to be done:* "When I am evaluating an early-career candidate, I want to see a system they architected and ran themselves — including what they chose not to build and why — so I can judge their engineering judgment rather than their certificate list."
3. **Problems being solved:**
 1. *For the viewer:* international catalogs are fragmented and poorly surfaced; regional content is hard to browse deliberately.
 2. *For the evaluator:* candidate portfolios typically show either tutorial-grade applications with no infrastructure, or architecture diagrams with nothing running behind them. Neither demonstrates operational ability.
 3. *For the author:* cloud costs make genuinely distributed practice projects unaffordable at student scale — which is itself the problem this architecture solves.
4. **How we know these problems exist (evidence):** the author's own job-search experience, where architecture knowledge was provable but hands-on deployment was not; job postings that explicitly require hands-on cloud and Kubernetes experience; and the sustained popularity of international titles such as *Squid Game* and *Stranger Things* in everyday watchlists, indicating real demand for cross-region catalogs. *Gap to close: no viewer interviews have been conducted yet — currently inferred demand, not researched demand.*

Solution:

1. **Brief:** A **hybrid-cloud** streaming catalog. A self-hosted three-node Kubernetes cluster runs the application — catalog browsing, Cognito-backed sign-in, per-profile favorites, rule-based recommendations, and an admin panel. **RDS PostgreSQL remains the system of record** (already provisioned, populated from TMDB and snapshotted; migrating it would cost effort without saving meaningful money), reached from the cluster over a secure link, with the existing **bastion EC2** retained for administrative access. **Cognito** handles authentication on its free tier, with the Cognito **sub** linking to existing profile and favorites tables. Trailers play as embedded YouTube videos and poster artwork is fetched by the browser from TMDB's own image hosting — the database stores URLs, not media files. The home origin is exposed through a free Cloudflare Tunnel, so no inbound ports are opened and the home IP is never published.
2. **Explicitly out of scope, with reasons:** video storage and streaming (cost and licensing); **CloudFront and any CDN of our own** — considered and rejected, because with all media third-party hosted and no cacheable origin assets, a CDN would front nothing and prove nothing while adding a cost line; multi-AZ and multi-region redundancy (no longer applicable once compute left AWS); and any ML-trained recommender (no interaction data, and prohibited by TMDB's terms).
3. Other alternatives are considered with prioritization metrics.

Decision 1: Where does the cluster live?

The core architectural fork. Scored 1-5, where 5 is best for that criterion.

Option	What it is	Running cost	Learning value	Demo impact	Ops overhead	Decision
A. Fully AWS-hosted (EKS)	Managed Kubernetes, RDS, ALB, NAT — everything rented	1 — control plane bills hourly regardless of traffic; ALB + NAT add fixed cost	5	5	2	 Rejected on cost
B. Fully self-hosted	3-node home cluster (k3s/kubeadm), 1 GPU node, MetalLB, Postgres on-node	5 — hardware is sunk cost; marginal cost is electricity	5 — bare-metal k8s, networking, storage all hands-on	3 — no managed-service integration to show	1 — everything is yours to run	 Migrating the existing RDS buys nothing
C. Hybrid — self-hosted origin + rented edge	Home cluster serves the app; RDS, bastion EC2 and Cognito remain in AWS, with a free Cloudflare Tunnel exposing the origin	4 — only RDS and the bastion bill; Cognito and the tunnel are free tier	5 — bare-metal k8s <i>and</i> hybrid networking, tunnelling, managed-auth integration	4 — a real system, deployed and operated end to end; no global edge to show	2 — cluster ops plus origin exposure to solve	 Chosen

Rationale: Option C moves the always-on hourly charges — EKS control plane, ALB, NAT gateway — onto hardware already being purchased, while leaving in AWS only what is already provisioned and cheap to keep: RDS as the system of record, the bastion for administrative access, and Cognito on its free tier. A CDN was evaluated as part of this option and dropped: with trailers embedded from YouTube and posters served from

TMDB's own hosting, there is no cacheable origin asset for an edge to front. The accepted trade-offs are that the home origin must be exposed safely (handled by a free Cloudflare Tunnel, outbound-only), that home-lab uptime is now the project's own responsibility, and that distant viewers are served from a single Seattle origin.

Decision 2: Container orchestration (retained for context)

Evaluated before the hybrid shift. Kept because the reasoning still explains the on-cluster choice.

Option	What it is	Ops overhead	Cost	Learning value	Decision
Plain EC2 / bare VMs	Install Docker, run containers manually; no orchestration	2 — manual restarts, patching, LB wiring	3	2	✗
ECS on EC2	AWS-native orchestrator on instances you own and patch	3	3	3 — AWS-proprietary, not portable	✗
ECS Fargate	Serverless containers; declare CPU/memory, no hosts	5 — lowest overhead of all options	3 — ~\$12 per 3 weeks for 0.5 vCPU / 1 GB, but ALB and NAT dominate the bill	2 — abstracts away the layer being learned	✗
Kubernetes (EKS → self-hosted k3s)	Open orchestration standard: pods, deployments, services	1 — highest; node upgrades, add-ons, version churn	5 — as self-hosted	5 — portable, industry-standard skill	✓ Chosen

Rationale: Kubernetes was chosen despite being the highest-overhead and initially the highest-cost option, because hands-on Kubernetes experience is the project's primary learning goal and the skill transfers across employers in a way ECS does not. Fargate was the strongest option on operational overhead and was rejected deliberately — it hides exactly the layer worth learning. Moving the cluster off AWS neutralizes the cost penalty that made this trade-off uncomfortable.

Decision 3: Video strategy

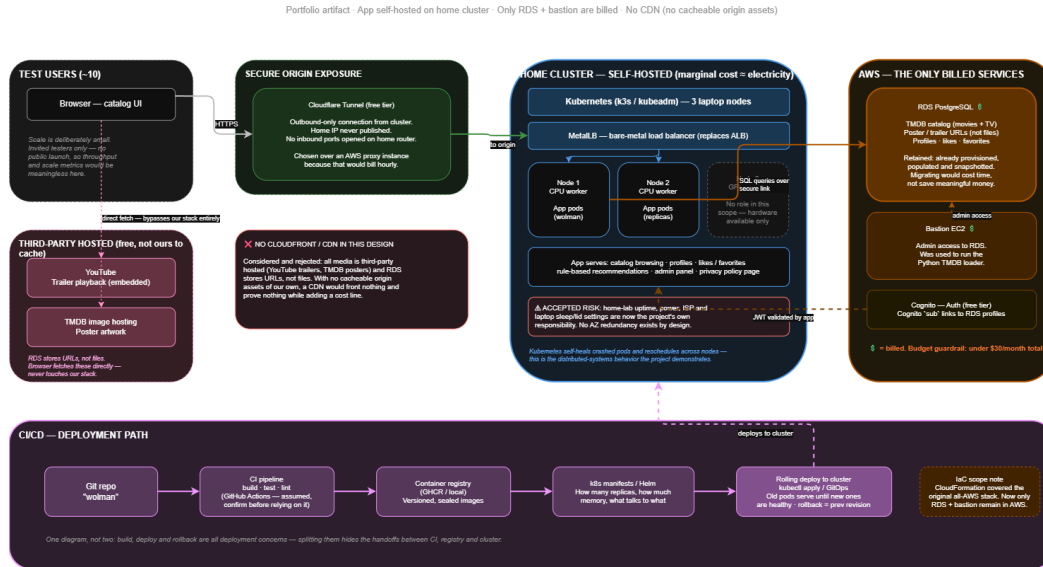
Option	Cost	Effort	Demo impact	Decision
Real video storage + streaming	1 — storage and egress scale badly	2	4	✗
Catalog only, embedded YouTube trailers	5 — no storage or egress cost	5	4 — proves the architecture without the media bill	✓ Chosen
Single dummy video for streaming proof	4	4	5 — demonstrates the streaming path exists	✓ Optional add-on

Decision 4: Recommendations

Option	Feasibility	Compliance	Effort	Decision
Trained ML model	1 — no meaningful user interaction data exists	1 — TMDB terms prohibit ML training/validation on their content	1	✗
Rule-based (genre/region affinity)	5	5	4	✓ Chosen

Rationale: Blocked on both data and licensing, not preference. Worth stating explicitly — reading the terms of service and designing around them is itself the product decision.

Architecture & deployment diagram



One diagram covering both runtime architecture and the CI/CD path, since build, deploy and rollback are all deployment concerns and splitting them hides the handoffs. Shows: source repo → build/registry → home cluster nodes (incl. the out-of-scope GPU node) → Cloudflare Tunnel → viewer, with Cognito and RDS PostgreSQL attached at their respective layers.

Product flow (Details of the feature/product):

1. Customer journey

The journey splits into two entry paths, which then converge on the same browse-to-play stretch.

Path A — First visit (new user)

1. **Land** on the site. Marketing surface explains what the catalog is.
2. **Sign up.** Account creation via Cognito. No browse-before-signup in this scope.
3. **Verify** the account (email verification through Cognito).
4. **Create a profile.** Name and avatar.
5. Converge on the browse stretch below.

Path B — Returning visit

1. **Land** on the site by typing the URL or opening a bookmark.
2. **Session check.** If the user previously chose "keep me signed in," the cached session is still valid and auth is skipped entirely.
3. **Profile picker** appears directly. The user selects a profile.
4. Converge on the browse stretch below.

If the session has expired or the user declined to stay signed in, Path B falls back to a sign-in screen before the profile picker.

The "stay signed in" decision

At sign-in the user explicitly chooses whether to remain signed in. This is a real user-facing decision and it drives token lifetime: a short-lived session for users who decline, a long-lived refresh token for users who accept. It is called out as its own journey step because it determines whether Path B skips authentication.

Shared stretch — Browse to play

1. **Catalog rows** render, organized by region and genre.
2. **Filter or scroll** by region or genre. The international catalog is large enough that filtering is a primary action, not a secondary one.
3. **Hover or click a poster.** A preview panel expands in place.
4. **Preview panel** shows a short synopsis, region, genre and year — enough to decide without leaving the row. This exists specifically to avoid a full page round trip for every title the user is merely curious about.
5. **Open the detail page** if still interested. Full synopsis, cast, metadata.
6. **Play.** The YouTube trailer embed launches.
7. **Add to favorites** from either the preview panel or the detail page.
8. **Return later** to the favorites list, which is the main re-entry point for returning users.

2. User stories

- **Explore by cast member.** As a viewer who has discovered an actor I like, I want to click their name in the cast list and see everything else they appear in, so that I can keep exploring from a person rather than starting a new search.
 - *Implementation note:* requires TMDB's person endpoint and a person detail page. Also requires cast to be stored as **linked person records**, not as a text blob on the title — cheap to do now, painful to retrofit later.

3. User acceptance criteria

Story: Explore by cast member

- **AC1 — Cast member is selectable.** *Given* I am on a title detail page (or expanded preview card) with a populated cast list, *when* I click an actor's name, *then* I am taken to that person's page.
- **AC2 — Other works are listed.** *Given* the selected person appears in other titles **within our catalog**, *when* their page loads, *then* every such title is displayed, excluding the title I navigated from.
- **AC3 — Partial filmography.** *Given* the person's full TMDB filmography is larger than what our catalog holds, *when* their page loads, *then* only the titles present in our catalog are shown. The catalog is a subset of TMDB by design; absent titles are a catalog limitation, not an error state.
- **AC4 — Empty state.** *Given* the person has no other titles in our catalog, *when* their page loads, *then* the page displays the message: *"This actor does not have any other works in our catalog."* — rather than an empty page with no explanation.

4. Edge cases

Context — how region works here. Region is derived from the viewer's IP address and determines which curated catalog shelf they see. This is **not** licensing enforcement: the author controls the catalog and could list anything. It is **catalog curation** — each region's shelf is deliberately sized and selected so the regional experience is meaningful, rather than every region showing the same undifferentiated pile of scraped titles. Regional distinctness is the product value being protected.

- **EC1 — VPN or proxy detected.** *Given* a viewer connects from an IP that resolves to a datacenter or known proxy range, *when* their region is determined, *then* access is blocked with an explanation of why, rather than silently serving a wrong shelf. Detection relies on a third-party IP intelligence database (e.g. IPinfo, MaxMind); consumer VPNs exit from known hosting ranges.
- **EC2 — False positive on VPN detection.** *Given* a legitimate viewer is behind a corporate network, university network or carrier-grade NAT that resolves to a datacenter range, *when* they are incorrectly flagged, *then* they are blocked despite being a real user. **Accepted limitation:** IP-based VPN detection is never fully accurate — it will block some legitimate viewers and miss some VPNs. A manual appeal or override path is out of scope.
- **EC3 — Geolocation fails entirely.** *Given* the viewer's IP cannot be resolved to a country, *when* the catalog loads, *then* the system falls back to a default global shelf rather than erroring or showing an empty catalog.
- **EC4 — TMDb is unavailable.** *Given* TMDb's API or image hosting is down or times out, *when* a page requiring live data loads, *then* the user sees a stated error rather than an indefinite spinner. Catalog metadata is already stored in RDS, so browsing degrades gracefully; live person-lookup and poster images are the exposed surfaces.
- **EC5 — Oversized filmography.** *Given* a selected cast member has a very large number of credits, *when* their person page loads, *then* results are paginated or capped rather than rendering unbounded.
- **EC6 — Missing or empty cast data.** *Given* a title has no cast data populated, *when* its detail page loads, *then* the cast section is omitted rather than rendering an empty container.

Known characteristic (not an edge case): with a single self-hosted origin in Seattle and no CDN, distant viewers experience higher latency. There is no fallback region to route to — this is an accepted consequence of the hybrid-cloud cost decision, not a defect with a defined behavior.

Tentative Timelines:

Task	Date
PRD finalized (solo project — self-approved)	09/25/2026 — done
Front end built	done
RDS provisioned and populated from TMDb	done
Week 1 — Hardware purchased; 3-node cluster built (k3s, MetalLB); Cloudflare Tunnel live	09/26/2026 – 10/02/2026
Week 2 — App containerized and deployed to cluster; Cognito auth wired; cluster reaches RDS	10/03/2026 – 10/09/2026
Week 3 — CI/CD pipeline; cast/person pages; region	10/10/2026 – 10/16/2026

shelves; node-failure and rollback tested	
Beta launch to ~10 invited testers	10/16/2026

Dependencies:

1. Open questions

Genuinely unresolved as of this version. Listed so they are decided deliberately rather than by accident mid-build.

- **Do we block VPNs at all?** This reversed twice while drafting: first tolerate VPN users and honor whatever IP is seen, then block them to keep regional shelves distinct. Both are defensible. Blocking costs a third-party IP intelligence dependency and guarantees some false positives; tolerating costs the regional distinctness the curation is meant to create. Unresolved.
- **Is region IP-only, or can the user override it?** A manual region picker would make VPN use unnecessary and sidestep the question above entirely — but it also makes regional shelves a preference rather than a place, which may weaken the premise.
- **How many titles per regional shelf counts as "curated"?** The shelf is deliberately sized rather than exhaustive, but no number has been set. Too few looks empty; too many defeats the purpose.
- **Does the GPU node get a real job, or stay decorative?** Currently out of scope and greyed out in the architecture diagram. Candidate use: an AI-engineering feature running on-cluster, which would make the node load-bearing rather than ornamental. No decision yet — and TMDB's terms constrain what can be trained on their content.
- **Which IP intelligence provider, and does a free tier cover this volume?** Only relevant if VPN blocking is adopted.

2. Infrastructure Requirements

Compute (self-hosted)

- 3 physical nodes purchased and assembled (1 with GPU, currently out of scope).
- Base OS installed and configured on each node; static addressing; SSH access.
- Kubernetes installed (k3s or kubeadm) — control plane plus workers.
- Application containerized; Docker images built for each service.
- Container registry for images to be pushed to and pulled from.

Networking

- Cloudflare Tunnel configured on-cluster — outbound-only, no inbound ports opened, home IP never published.
- MetalLB for in-cluster load balancing across nodes.
- DNS records pointing at the tunnel endpoint.
- Secure link from cluster to RDS.

Storage

- Persistent volumes for any stateful workload — pods are ephemeral, so anything that must survive a restart needs backing storage.

- Backup path for RDS snapshots.

AWS (already provisioned)

- RDS PostgreSQL — system of record, populated from TMDB. Billed.
- Bastion EC2 — administrative access to RDS. Billed.
- Cognito — authentication, free tier.

Physical / environmental

- Reliable power and home network capacity for three always-on nodes.
- Nodes must stay awake — sleep and lid-close behavior disabled where applicable.

3. Budget Approvals

Solo project — the author is both requester and approver. There is no external sign-off; the budget is a self-imposed constraint, which is the point of stating it.

One-time capital

- 3 nodes (one with GPU) — purchased out of pocket. Treated as sunk cost: the hardware outlives this project and is reusable for future work.

Recurring

- AWS: hard ceiling of \$30 per month, with RDS and the bastion EC2 as the only billed services. Cognito stays on the free tier.
- Cloudflare Tunnel: free tier.
- Electricity for three always-on nodes — not separately metered; accepted as household cost.
- Optional, only if VPN blocking is adopted: an IP intelligence provider. Must fit a free tier or it does not proceed.

Approval rule: no new recurring charge is added without displacing an existing one. Any spend that bills hourly regardless of traffic is rejected by default — that pattern is what drove the architecture off AWS in the first place.

4. Partner Support (APIs & Partnerships)

No formal partnerships. Two external providers the system depends on:

- **AWS** — RDS, EC2 bastion, Cognito. Standard account, no support plan.
- **TMDB** — catalog metadata, poster image URLs, cast and person data. Free API key under their terms of use: attribution required, no profit from the deployment, no ML training on their content.

5. Internal Dependencies (Tech, Design, Other Teams)

None. This is a solo project: product, design, engineering and operations are the same person, so there are no cross-team handoffs, no blocked reviews and no waiting on another team's roadmap.

Worth stating rather than leaving blank, because it changes how the rest of this document should be read. The work spans roles that would normally sit on separate teams — product decisions, interface design, infrastructure build, deployment and ongoing operations — and they are consolidated here by necessity, not by preference. The upside is that no decision is blocked on anyone else. The trade-off is that there is no second reviewer: every choice in this PRD went unchallenged except by the author.3. 4. sp5. dtdot

